

Rapport de projet IA echec

Martim Lopes, Marceau Cattet, Gabriel Bouchemal

SOMMAIRE :

1) INTRODUCTION

2) EXPLICATION DES FICHIERS

A) fichier IA : ai.py, evalu.py, opening_book.py

B) fichier interface graphique : gui.py, training_gui.py

C) fichier qui gère les règles : engine_chess.py

D) fichier de système d'entraînement : weights.py, selfplay_train.py, reset_training.py, plot_progress.py

E) fichier Main : main.py, main_train.py

3) CONCLUSION, DIFFICULTES, AXE D'AMELIORATION

1° INTRODUCTION:

Ce projet de NSI consiste en la création d'un moteur d'échecs complet en python, accompagné d'une intelligence artificielle capable de jouer de manière autonome et de s'améliorer par elle-même au fil du temps.

Le projet permet de jouer aux échecs contre une IA, d'observer deux IA jouer l'une contre l'autre, et d'entraîner cette IA pour qu'elle devienne progressivement plus forte. Concrètement trois choses sont possibles :

Lancer `main.py` ouvre l'interface de jeu. Le joueur peut affronter l'IA ou regarder deux IA s'affronter, avec un panneau d'analyse en temps réel qui affiche le score de la position, les cinq meilleurs coups envisagés et les statistiques de calcul (nombre de positions analysées, temps, vitesse).

Lancer `main_train.py` ouvre l'interface d'entraînement. celle-ci permet de lancer des sessions ou l'IA joue contre des versions légèrement modifiées d'elle même, garde les modifications qui améliorent ses performances et sauvegarde chaque génération. On peut aussi faire s'affronter deux versions différentes de l'IA en match direct pour comparer leurs forces.

Lancer `selfplay_train.py` en ligne de commande fait la même chose sans interface, idéal pour des entraînements longs en arrière plan.

Le projet est organisé en cinq grandes parties distinctes.

La première partie regroupe les fichiers de l'intelligence artificielle : `ai.py` qui contient l'algorithme de recherche Negamax avec élagage Alpha-Beta pour choisir le meilleur coup, `evalu.py` qui attribue un score numérique à chaque position selon sept critères stratégiques (matériel, centre, développement, sécurité du roi, mobilité, structure de pions, activité du roi en finale), et `opening_book.py` qui fournit des débuts de parties préenregistrés pour les premiers coups.

La deuxième partie regroupe les fichiers d'interface graphique : `gui.py` qui gère l'échiquier, les interactions du joueur et le panneau d'analyse en temps réel, et `training_gui.py` qui fournit l'interface permettant de lancer et suivre les sessions d'entraînement.

La troisième partie est constituée d'un seul fichier, `engine_chess.py`, qui gère toutes les règles officielles des échecs en s'appuyant sur la bibliothèque python chess (coups légaux, roque, prise en passant, promotion, échec et mat, pat, règle des 50 coups).

La quatrième partie regroupe les fichiers du système d'entraînement : `weights.py` qui définit les 40 paramètres numériques de la fonction d'évaluation, `selfplay_train.py` qui implémente l'algorithme évolutionnaire en ligne de commande, `reset_training.py` qui permet de tout réinitialiser proprement et `plot_progress.py` qui génère un graphique de la progression de l'IA au fil des générations.

La cinquième partie regroupe les fichiers main : `main.py` qui est le point d'entrée du jeu et `main_train.py` qui est le point d'entrée de l'interface d'entraînement

Le projet est entièrement écrit en python et repose sur trois bibliothèques principales : `python-chess` pour la gestion des règles et des positions, `Tkinter` pour les deux interfaces graphiques (jeu et entraînement), et le module standard `threading` pour faire tourner les calculs de l'IA en parallèle de l'interface sans la bloquer. Les données d'entraînement sont sauvegardées au format JSON pour les poids et JSONL pour l'historique des générations

2° EXPLICATION DES FICHIERS

A) fichier IA :

--fichier `ai.py` :

Utilité et fonctionnement :

C'est le fichier le plus important du projet. C'est lui qui décide quel coup jouer à chaque tour. Quand c'est au tour de l'IA, ce fichier analyse toutes les possibilités disponibles, les évalue, et retourne le coup qu'il juge le meilleur. Il est utilisé aussi bien pendant le jeu normal que pendant l'entraînement, où deux versions de l'IA s'affrontent

Le fonctionnement repose sur un principe simple : explorer les coups possibles en avance, comme un joueur humain qui réfléchit "si je joue ici, il va répondre là, et alors je peux faire ceci...". Plus on explore loin dans le futur, plus l'IA joue bien, mais plus c'est lent.

Explications du code :

La fonction `set_active_weights(w)` sert pendant l'entraînement. Quand deux versions de l'IA s'affrontent, il faut pouvoir changer les paramètres d'évaluation entre les deux joueurs. Cette fonction injecte le bon dictionnaire de poids dans l'évaluateur avant chaque coup.

```
def set_active_weights(w: dict) -> None:
    """Appelé par le training pour injecter les poids du candidat en cours."""
    global _ACTIVE_WEIGHTS
    _ACTIVE_WEIGHTS = dict(w)
```

La fonction `_resolve_weights(w_arg)` est une fonction de priorité. Elle décide quels poids utiliser dans cet ordre : d'abord ceux passés en argument, sinon les poids actifs injectés par l'entraînement, sinon les meilleurs poids sauvegardés sur le disque, et en dernier recours les valeurs par défaut. Ça garantit que l'IA a toujours des poids valides peut importe le contexte.

```
def _resolve_weights(w_arg) -> dict:
```

La fonction `get_history_uci(board)` retourne simplement la liste de tous les coups déjà joués dans la partie au format UCI (ex: e2e4). Utilisée pour savoir si on est encore dans le livre d'ouverture.

```
def get_history_uci(board: chess.Board) -> list[str]:
    """retourne la liste des coups déjà joués dans la partie , au format UCI"""

    return [m.uci() for m in board.move_stack]
```

La fonction `evaluate_board(board, weights)` est un wrapper qui appelle la vraie fonction d'évaluation de `evalu.py` en lui injectant les poids résolus. Elle sert de point d'entrée unique pour tout ce qui touche à l'évaluation dans `ai.py`.

```
def evaluate_board(board, weights=None):
    """Wrapper qui injecte les poids résolus dans l'évaluateur."""
    return _evaluate_board_raw(board, _resolve_weights(weights))
```

La classe `SearchResult` est un simple conteneur de données. Elle stocke le résultat d'une recherche : le meilleur coup trouvé, son score et le nombre de noeuds explorés. Elle est retournée par `choose_best_move` et `choose_best_move_timed` pour transmettre toutes ces informations en une seule fois.

```
@dataclass
class SearchResult:
    # Cette classe contient le résultat final de la recherche.
    move: Optional[chess.Move]
    score: float
    nodes: int
```

La fonction `_age_history()` gère la table HISTORY qui accumule des scores pour les coups efficaces au fil du temps. Sans nettoyage, les vieilles parties influenceraient trop les nouvelles. Cette fonction divise tous les scores par 2 et supprime ceux qui tombent à 0, comme un mécanisme d'oubli progressif.

```
def _age_history():
```

La fonction `_update_killer(depth, move)` enregistre un coup non-capture qui a provoqué une coupure beta comme killer move pour cette profondeur. La fonction garde au maximum 2 killer moves par profondeur : le nouveau coup est inséré en première position et si la liste dépasse 2 entrées le plus ancien est supprimé.

```
def _update_killer(depth: int, move: chess.Move):
```

La fonction `_update_history(piece_type, to_square, depth)` récompense un coup efficace dans la table HISTORY. Plus la profondeur où il a causé une coupure est grande plus la récompense est élevée ($depth * depth$). Le score est plafonné à HISTORY_MAX pour éviter que certains coups dominent trop.

```
def _update_history(piece_type: int, to_square: int, depth: int):
```

La fonction `_move_score(board, move, depth, tt_best)` attribue un score de priorité à chaque coup pour déterminer l'ordre d'exploration. Les coups sont classés ainsi : coup de la table de transposition en premier, puis les captures classées par MVV-LVA (valeur capturée - valeur attaquant), puis les promotions, puis les killer moves, puis les coups qui donnent échec, et enfin les coups historiquement efficaces.

```
def _move_score(board: chess.Board, move: chess.Move, depth: int,
                tt_best: Optional[chess.Move]) -> int:
```

La fonction `quiescence(board, alpha, beta, nodes_ref)` est appelée quand la recherche principale atteint profondeur 0. On ne s'arrête pas si des captures sont en cours : elle continue d'explorer uniquement les captures et promotions pour éviter l'effet horizon, c'est à dire rater une combinaison tactique juste parce qu'on s'est arrêté au mauvais moment. Elle inclut aussi un delta pruning : si même la meilleure capture possible ne suffit pas à améliorer alpha, on abandonne directement.

```
def quiescence(board: chess.Board, alpha: float, beta: float,
               nodes_ref: list, qdepth: int = 0,
               weights=None) -> float:
```

La fonction `negamax(board, depth, alpha, beta, nodes_ref, ply)` est le coeur du moteur. Elle explore récursivement tous les coups jusqu'à la profondeur demandée. A chaque appel elle vérifie d'abord la table de transposition, gère les cas terminaux (mat, pat), applique le null move pruning, trie les coups, et pour chaque coup joue le coup, rappelle `negamax` avec `depth-1`, annule le coup et compare le score. Elle applique aussi le Late Move Reduction : les coups tardifs peu prometteurs sont explorés moins profondément pour gagner du temps.

```
def negamax(board: chess.Board, depth: int, alpha: float, beta: float,
            nodes_ref: list, ply: int = 0,
            allow_null: bool = True,
            weights=None) -> tuple[float, Optional[chess.Move]]:
```

La fonction `alphabeta(board, depth, alpha, beta, ply, tt, weights)` est une simple fonction d'interface qui appelle `negamax` et reformate le résultat pour le rendre compatible avec `gui.py`. Elle existe uniquement pour que le GUI n'ait pas besoin de connaître les détails internes de `negamax`.

```
def alphabeta(board: chess.Board, depth: int, alpha: float, beta: float,
              ply: int, tt: dict, weights: Any) -> tuple:
```

La fonction `evaluate(board, ply, weights)` est une interface simplifiée vers la fonction d'évaluation de `evalu.py`. Appelée quand on a juste besoin du score d'une position sans lancer une recherche complète.

```
def evaluate(board: chess.Board, ply: int = 0, weights: Any = None) -> float:
```

La fonction `order_moves(board, moves, tt_best, killer_moves, level)` trie une liste de coups en utilisant `_move_score`. Utilisée depuis l'extérieur du fichier quand on a besoin d'un tri de coups sans lancer `negamax`.

```
def order_moves(board: chess.Board, moves, tt_best=None,
                killer_moves=None, level=None):
```

La fonction `choose_best_move(board, depth, weights)` est la version simple de la recherche : elle lance `negamax` à profondeur fixe. Elle vérifie d'abord le livre d'ouverture, et si on est dans les 6 premiers coups retourne directement le coup du livre sans calculer. Sinon elle lance `negamax` et retourne un `SearchResult` avec le meilleur coup, son score et le nombre de noeuds explorés.

```
def choose_best_move(board: chess.Board, depth: int,
                     weights: Any = None) -> SearchResult:
```

La fonction `choose_best_move_timed(board, movetime_ms, max_depth, weights)` est la version avec limite de temps. Elle augmente la profondeur de 1 en 1 jusqu'à ce que le temps soit écoulé. A partir de la profondeur 3 elle utilise les aspiration windows : on cherche d'abord dans une fenêtre étroite autour du score précédent, et si le résultat sort de cette fenêtre on relance avec une fenêtre pleine. Ca permet de trouver le même résultat plus rapidement dans la majorité des cas.

```
def choose_best_move_timed(board: chess.Board, movetime_ms: int,
                           max_depth: int = 6,
                           weights: Any = None) -> SearchResult:
```

La fonction `choose_move(board)` est la plus simple du fichier. Elle appelle juste `choose_best_move` avec la profondeur par défaut. Utile quand on veut un coup rapidement sans se soucier des paramètres

```
def choose_move(board: chess.Board) -> Optional[chess.Move]:
```

--fichier `evalu.py` :

`evalu.py` est le fichier qui permet à l'IA de juger si une position est bonne ou mauvaise. Sans ce fichier, `negamax` ne pourrait pas comparer les positions entre elles et choisir un coup. Concrètement il attribue un score numérique à chaque position : un score positif signifie que les noirs sont avantagés, un score négatif que les blancs sont avantagés.

Pour calculer ce score il ne regarde pas qu'une seule chose. Il analyse sept critères différents : le matériel (qui a le plus de pièces), le contrôle du centre, le développement des pièces en ouverture, la sécurité du roi, la mobilité (nombre de coups disponibles), la structure des pions et l'activité du roi en finale. Chaque critère est multiplié par un coefficient qui varie selon la phase de jeu, ce qui permet à l'IA de jouer différemment en ouverture, en milieu de partie et en finale.

Tous ces coefficients sont les 40 paramètres de `weights.py`. C'est ce qui rend le fichier entraînable : en modifiant ces valeurs on change la façon dont l'IA évalue les positions.

Explications du code :

La fonction `_get_piece_values(w)` construit le dictionnaire qui associe chaque type de pièce à sa valeur en centipions à partir des poids. Elle utilise un cache : si les mêmes valeurs ont déjà été construites pour ces poids, elle retourne directement le résultat mémorisé sans le recalculer. Ca évite de refaire ce travail à chaque évaluation.

```
def _get_piece_values(w: Dict[str, float]) -> dict:
```


La fonction `game_phase(board, w)` détermine dans quelle phase de jeu on se trouve en comptant le nombre total de pièces mineures et majeures restantes sur le plateau. Si ce nombre dépasse le seuil d'ouverture on est en ouverture, si il dépasse le seuil de finale on est en milieu de jeu, sinon on est en finale. Ces deux seuils font partie des paramètres entraînables.

```
def game_phase(board: chess.Board, w: Dict[str, float]) -> str:
```

La fonction `_material_score(board, pv)` calcule la différence de matériel entre les deux camps. Elle additionne la valeur de toutes les pièces noires, additionne la valeur de toutes les pièces blanches, et retourne la différence. Un résultat positif signifie que les noirs ont plus de matériel.

```
def _material_score(board: chess.Board, pv: dict) -> float:
```

La fonction `_center_score(board, w)` évalue le contrôle du centre en comptant combien de cases centrales sont attaquées par chaque camp. Elle distingue le centre principal (d4, e4, d5, e5) et le centre élargi, chacun avec son propre coefficient.

```
def _center_score(board: chess.Board, w: Dict[str, float]) -> float:
```

La fonction `_development_score(board, w)` mesure le développement en comptant combien de cavaliers et de fous ont quitté leur rangée de départ pour chaque couleur. Une pièce mineure qui n'a pas bougé depuis le début est considérée comme non développée. Ce critère n'est utilisé qu'en ouverture et en milieu de jeu. Elle contient la méthode interne `developed(color)` qui fait le comptage pour une couleur donnée.

```
def _development_score(board: chess.Board, w: Dict[str, float]) -> float:
[...]  
    def developed(color):  
        [...]
```

La fonction `_king_safety_score(board, w)` évalue la sécurité du roi en regardant combien de cases autour de lui sont attaquées par l'adversaire, et si le droit au roque est encore disponible. Elle contient la méthode interne `safety(color)` qui calcule le score de sécurité pour une couleur donnée.

```
def _king_safety_score(board: chess.Board, w: Dict[str, float]) -> float:
[...]  
    def safety(color):  
        [...]
```

La fonction `_mobility_score(board, w)` compare le nombre de coups légaux disponibles pour chaque camp. Pour ce faire elle change temporairement le tour dans le plateau, compte les coups légaux de chaque côté, puis remet le tour à sa valeur d'origine. Plus un joueur a de coups disponibles plus sa position est considérée comme dynamique.

```
def _mobility_score(board: chess.Board, w: Dict[str, float]) -> float:
```

La fonction `_pawn_structure_score(board, w)` analyse la qualité des pions des deux camps. Elle penalise les pions doublés (deux pions sur la même colonne), les pions isolés (aucun pion ami sur les colonnes adjacentes), et récompense les pions passés (aucun pion adverse devant eux) avec un bonus qui augmente selon leur avancement. Elle contient la méthode interne `score_for(color)` qui fait toute l'analyse pour une couleur donnée.

```
def _pawn_structure_score(board: chess.Board, w: Dict[str, float]) -> float:
```

La fonction `score_for(color)` est une méthode interne de `_pawn_structure_score`. Elle fait tout le travail d'analyse des pions pour une couleur donnée. Elle parcourt chaque colonne du plateau, regarde combien de pions sont dessus pour détecter les pions doublés, puis pour chaque pion elle vérifie s'il est passé en regardant si aucun pion adverse ne se trouve devant lui sur sa colonne et les colonnes adjacentes, et vérifie s'il est isolé en regardant si aucun pion ami ne se trouve sur les colonnes voisines. Elle retourne le score total des pions pour cette couleur avec les bonus et pénalités accumulés.

```
def score_for(color):
```

La fonction `_endgame_king_score(board, w)` est uniquement appelée en finale. Elle récompense le roi qui est actif (qui attaque beaucoup de cases) et bien centralisé. En finale un roi au centre est un atout majeur contrairement à l'ouverture où il doit rester protégé. Elle contient la méthode interne `king_score(color)` qui calcule le score pour un roi donné.

```
def _endgame_king_score(board: chess.Board, w: Dict[str, float]) -> float:
```

La fonction `evaluate_board(board, weights)` est la fonction principale du fichier, celle qui est appelée depuis l'extérieur. Elle commence par gérer les cas terminaux (mat et pat), détermine la phase de jeu, sélectionne les bons multiplicateurs pour cette phase, puis additionne tous les sous scores en les multipliant par leurs coefficients respectifs. C'est elle qui combine tout le travail des autres fonctions en un seul score final.

```
def evaluate_board(board: chess.Board,  
                  weights: Optional[Dict[str, float]] = None) -> float :
```

--fichier opening_book.py :

Utilisation et fonctionnement :

opening_book.py est le fichier qui permet à l'IA de jouer les premiers coups de la partie sans avoir à calculer. En effet lancer negamax dès le premier coup serait inutile car les ouvertures aux échecs sont très bien connues et étudiées depuis des siècles. Ce fichier contient donc une collection de lignes d'ouverture préenregistrées que l'IA suit automatiquement pendant les 6 premiers demi coups.

Exemple d'ouverture possible :

```
# Italienne classique
["e2e4", "e7e5", "g1f3", "b8c6", "f1c4"],

# Scotch
["e2e4", "e7e5", "g1f3", "b8c6", "d2d4"],

# Sicilienne ouverte
["e2e4", "c7c5", "g1f3", "d7d6", "d2d4"],

# Française avance
["e2e4", "e7e6", "d2d4", "d7d5", "e4e5"],

# Gambit Dame refusé
["d2d4", "d7d5", "c2c4", "e7e6", "b1c3"],
```

Le fonctionnement est simple : au premier coup l'IA choisit aléatoirement une ligne parmi celles disponibles pour sa couleur. Elle mémorise cette ligne dans CURRENT_LINE et la suit coup après coup tant que l'adversaire joue dans le même sens. Si l'adversaire s'écarte de la ligne prévue, le livre est abandonné et l'IA reprend ses calculs normaux. Les lignes sont organisées par catégorie : une pour les blancs, et plusieurs pour les noirs selon le premier coup adverse (contre e4, contre d4, contre c4, contre f3).

Explications du code :

La fonction reset_line() remet simplement CURRENT_LINE à None. Elle est appelée au début de chaque nouvelle partie depuis new_game() dans gui.py, ou quand la position s'écarte de la ligne en cours. Sans cet appel l'IA continuerait à chercher sa ligne de la partie précédente au début de la nouvelle.

```
def reset_line():
```

La fonction `pick_book_move(history, is_white)` est la fonction principale du fichier. Elle reçoit la liste des coups déjà joués et un booléen indiquant si c'est au tour des blancs. Si aucune ligne n'est encore choisie elle en sélectionne une aléatoirement dans la bonne catégorie selon la couleur et le premier coup adverse. Si une ligne est déjà en cours elle vérifie que l'historique correspond bien au début de cette ligne, et si c'est le cas retourne le prochain coup à jouer. Si l'historique ne correspond plus ou si on dépasse les 6 demi coups elle appelle `reset_line()` et retourne `None` pour signaler qu'on sort du livre.

```
def pick_book_move(history: List[str], is_white: bool) -> str | None:
```

Ainsi le groupe IA est composé de trois fichiers qui travaillent ensemble dans un ordre précis. Tout commence par `evalu.py` qui est la base : il sait lire un plateau et lui donner un score. Il ne sait pas jouer, il sait juste dire si une position est bonne ou mauvaise. `opening_book.py` lui est complètement indépendant, il contient juste une liste de coups préenregistrés pour les débuts de partie.

`ai.py` est celui qui orchestre tout. Quand il doit choisir un coup il commence par consulter `opening_book.py` pour voir si on est encore dans les premières lignes connues. Si oui il retourne directement le coup du livre sans calculer. Sinon il lance `negamax` qui va explorer des milliers de positions en jouant et annulant des coups. A chaque fois qu'il atteint la profondeur maximale il appelle `evalu.py` pour savoir si la position est bonne ou mauvaise. C'est ce score qui remonte dans `negamax` pour choisir le meilleur coup. Au final `ai.py` retourne un `SearchResult` avec le meilleur coup trouvé.

B) fichier interface graphique

--fichier `gui.py` :

Utilité et fonctionnement :

`gui.py` est le fichier qui gère tout ce que l'utilisateur voit et avec quoi il interagit. C'est lui qui affiche l'échiquier, traite les clics du joueur, orchestre les parties et affiche le panneau d'analyse en temps réel. Sans ce fichier le moteur tournerait mais on ne pourrait pas jouer. Il fait le lien entre le joueur humain, l'IA et le moteur de règles.

Le fichier est organisé en deux grandes classes. `AnalysisPanel` gère le panneau latéral qui affiche le score de la position, les statistiques de calcul et les cinq meilleurs coups envisagés par l'IA. `ChessApp` gère tout le reste : l'échiquier, les modes de jeu, les interactions utilisateur et la coordination entre tous les modules du projet.

Explications du code :

La classe AnalysisPanel est le panneau d'analyse affiché à droite de l'échiquier. Elle contient tous les widgets qui montrent les informations de calcul de l'IA en temps réel.

```
class AnalysisPanel(tk.Frame):
```

La methode `init(self, parent)` de la classe AnalysisPanel est le constructeur du panneau. Elle crée et organise tous les widgets : le label de score, la barre graphique blanc noir, les labels de statistiques et la zone scrollable des meilleurs coups.

```
def __init__(self, parent):
```

La methode `_safe_score(score)` de la classe AnalysisPanel sécurise une valeur de score en remplaçant les valeurs mathématiques invalides comme NaN ou infini par MATE_SCORE. Elle evite tout crash lors de l'affichage.

```
def _safe_score(score: float) -> float:
```

La methode `_score_text(score)` de la classe AnalysisPanel convertit un score numérique brut en texte lisible. Elle affiche Mat+2 si un mat est détecté, ou la valeur en pions comme +1.50 sinon.

```
def _score_text(score: float) -> str:
```

La methode `update_evaluation(self, score, perspective)` de la classe AnalysisPanel met à jour le label de score et la barre graphique après chaque calcul de l'IA. Elle choisit aussi la couleur du texte selon l'avantage : vert si bon, rouge si mauvais, gris si nul.

```
def update_evaluation(self, score: float, perspective: int = 1):
```

La methode `_draw_bar(self, score)` de la classe AnalysisPanel dessine la barre bicolore blanc noir sur le canvas. Le score est limité entre -10 et +10 pour le dessin, le centre représente l'égalité.

```
def _draw_bar(self, score: float):
```

La methode `update_stats(self, nodes, elapsed)` de la classe AnalysisPanel met à jour les trois labels de statistiques : nombre de noeuds explorés, temps de calcul et noeuds par seconde.

```
def update_stats(self, nodes: int, elapsed: float):
```

La methode `update_top_moves(self, moves_data, chosen_uci)` de la classe `AnalysisPanel` efface les anciens widgets de coups et recrée la liste des cinq meilleurs coups. Elle surligne en vert et ajoute une coche sur le coup effectivement joué par l'IA.

```
def update_top_moves(self, moves_data: List[Dict],
                    chosen_uci: Optional[str] = None):
```

La methode `clear(self)` de la classe `AnalysisPanel` remet tout le panneau à zéro : score à 0.00, stats vides, barre centrée et liste de coups vide. Appelée au début de chaque nouvelle partie.

```
def clear(self):
```

La classe `ChessApp` est la classe principale de l'application. Elle gère l'échiquier, les modes de jeu, les interactions utilisateur et coordonne tous les modules du projet.

```
class ChessApp:
```

La methode `__init__(self, root)` de la classe `ChessApp` est le constructeur principal. Elle initialise toutes les variables de l'application, construit l'interface et ouvre le dialogue de choix de mode après 100 millisecondes.

```
def __init__(self, root: tk.Tk):
```

La methode `_build_interface(self)` de la classe `ChessApp` construit toute la mise en page : colonne gauche avec le statut, l'échiquier et les contrôles, et colonne droite avec le panneau d'analyse.

```
def _build_interface(self):
```

La methode `_build_board(self)` de la classe `ChessApp` crée les 64 boutons de l'échiquier en grille 8x8. Chaque bouton est relié à `on_click` avec ses coordonnées.

```
def _build_board(self):
```

La methode `_ask_game_mode(self)` de la classe `ChessApp` ouvre une fenêtre modale proposant les deux modes de jeu. Elle contient les fonctions internes `hvai()` pour le mode humain contre IA et `avai()` pour le mode IA contre IA.

```
def _ask_game_mode(self):
```

La methode `_ask_side(self)` de la classe `ChessApp` ouvre une fenêtre modale pour choisir la couleur du joueur humain. Elle contient les fonctions internes `sw()` pour les blancs et `sb()` pour les noirs.

```
def _ask_side(self):
```

La methode `_center(win)` de la classe `ChessApp` centre une fenêtre `Toplevel` au milieu de l'écran en calculant les coordonnées depuis la résolution de l'écran.

```
def _center(win: tk.Toplevel):
```

La methode `_start_game(self)` de la classe `ChessApp` lance la partie : elle rafraichit l'affichage et si c'est à l'IA de commencer en premier elle declenche `ai_play` après 200 millisecondes.

```
def _start_game(self):
```

La methode `_start_ai_vs_ai(self)` de la classe `ChessApp` active les flags de la partie IA contre IA puis appelle `_start_game()`.

```
def _start_ai_vs_ai(self):
```

La methode `new_game(self)` de la classe `ChessApp` remet tout à zéro : arrête l'IA, réinitialise le plateau, la sélection, le dernier coup, réinitialise le livre d'ouverture, lit les nouvelles valeurs des spinboxes, vide le panneau d'analyse et rouvre le dialogue de mode.

```
def new_game(self):
```

La methode `toggle_pause(self)` de la classe `ChessApp` bascule entre pause et reprise en mode IA contre IA. Elle met à jour le texte du bouton et relance `_ai_vs_ai_step` si on reprend.

```
def toggle_pause(self):
```

La methode `on_click(self, r, c)` de la classe `ChessApp` gère les clics du joueur humain. Premier clic sur une pièce amie : la sélectionne et affiche les destinations légales en jaune. Deuxième clic sur une destination valide : joue le coup, rafraichit l'affichage et declenche `ai_play`.

```
def on_click(self, r: int, c: int):
```

La methode `_analyze_position(self, depth)` de la classe `ChessApp` est le coeur de l'analyse. Elle consulte d'abord le livre d'ouverture, et si on en sort elle évalue chaque coup légal en lançant `negamax` sur la position après ce coup, trie les résultats et retourne le meilleur score, la liste des coups, le nombre de noeuds et le temps écoulé.

```
def _analyze_position(self, depth: int):
```

La methode `ai_play(self)` de la classe `ChessApp` fait jouer l'IA un coup en mode humain contre IA. Elle appelle `_analyze_position`, choisit le meilleur coup, met à jour le panneau d'analyse, applique le coup et vérifie la fin de partie.

```
def ai_play(self):
```

La methode `_ai_vs_ai_step(self)` de la classe `ChessApp` fait jouer un coup en mode IA contre IA. Elle est identique à `ai_play` mais se reprogramme elle même via `root.after()` pour enchaîner les coups automatiquement avec le délai configuré.

```
def _ai_vs_ai_step(self):
```

La methode `_check_end(self)` de la classe `ChessApp` vérifie si la partie est terminée en testant quatre conditions : échec et mat, pat, matériel insuffisant et règle des 50 coups. Elle affiche un message et retourne `True` si la partie est finie.

```
def _check_end(self) -> bool:
```

La methode `_refresh(self)` de la classe `ChessApp` redessine les 64 cases de l'échiquier en mettant à jour les couleurs de fond et les symboles de pièces de chaque case

```
def _refresh(self):
```

La fonction `main()` crée la fenêtre Tkinter, instancie `ChessApp` et lance la boucle principale avec `root.mainloop()`

```
def main():
```

--fichier `training_gui.py` :

Utilité et fonctionnement : `training_gui.py` est le fichier qui fournit une interface graphique dédiée à l'entraînement de l'IA. Il permet de lancer des sessions d'évolution automatique des paramètres, de faire s'affronter deux versions de l'IA en match libre, et de consulter les statistiques des sessions passées. L'interface est organisée en trois onglets : Entraînement, Match et Statistiques. Pour ne pas bloquer l'interface pendant les calculs, tout l'entraînement tourne dans un thread séparé

Le fonctionnement repose sur deux classes. `TrainingThread` est le thread qui fait tourner l'entraînement en arrière plan : il mute les poids, lance les parties, compare les résultats et sauvegarde les meilleures générations. `TrainingGUI` est l'interface graphique qui permet à l'utilisateur de configurer les paramètres, de démarrer et arrêter l'entraînement, et de visualiser la progression en temps réel

Explications du code :

La classe `TrainingThread` est le thread d'entraînement. Elle hérite de `threading.Thread` ce qui lui permet de tourner en parallèle de l'interface sans la bloquer. Le paramètre `daemon=True` garantit qu'elle s'arrête automatiquement si la fenêtre est fermée.

```
class TrainingThread(threading.Thread):
```

La méthode `__init__(self, app, mode)` de la classe `TrainingThread` est le constructeur du thread. Elle initialise la référence à l'application, le mode (training ou match), et les drapeaux `running` et `paused`.

```
def __init__(self, app, mode):
```

La méthode `run(self)` de la classe `TrainingThread` est la méthode exécutée au démarrage du thread. Elle redirige vers `_run_training()` ou `_run_match()` selon le mode choisi.

```
def run(self):
```

La méthode `_run_training(self)` de la classe `TrainingThread` est la boucle principale d'entraînement. Pour chaque itération elle vérifie si le thread doit continuer ou est en pause, mute les poids avec `_mutate`, évalue le candidat avec `_eval_match`, accepte ou rejette le candidat selon le `winrate`, sauvegarde la nouvelle génération si acceptée et met à jour l'interface.

```
def _run_training(self):
```

La méthode `_run_match(self)` de la classe `TrainingThread` est la boucle principale de match libre. Elle fait jouer `N` parties entre les poids `A` et `B` en alternant les couleurs, affiche le score au fil des parties et annonce le vainqueur à la fin.

```
def _run_match(self):
```

La méthode `_mutate(self, w, step)` de la classe `TrainingThread` effectue la mutation des poids. Elle choisit aléatoirement un groupe de paramètres, mute 1 à 3 paramètres de ce groupe avec une amplitude proportionnelle à leur échelle, et respecte les bornes définies dans `WEIGHT_BOUNDS`.

```
def _mutate(self, w, step):
```

La methode `_eval_match(self, best, cand)` de la classe `TrainingThread` évalue un candidat contre le champion sur `games_per_eval` parties en alternant les couleurs. Elle calcule le winrate du candidat et retourne ce winrate ainsi qu'un dictionnaire contenant le nombre de victoires, nulles et défaites.

```
def _eval_match(self, best, cand):
```

La methode `_play_game(self, ww, wb)` de la classe `TrainingThread` joue une partie complète entre deux jeux de poids. Elle injecte les poids du joueur actif avant chaque coup, s'arrête en cas de fin de partie ou après `max_plies` demi coups, et retourne +1 si les blancs gagnent, -1 si les noirs gagnent et 0 pour toute nulle.

```
def _play_game(self, ww, wb):
```

La methode `stop(self)` de la classe `TrainingThread` met `running` à `False` ce qui fera terminer la boucle au prochain tour.

```
def stop(self):
```

La methode `pause(self)` de la classe `TrainingThread` met `paused` à `True` ce qui suspend la boucle via le `while self.paused` dans `_run_training`.

```
def pause(self):
```

La methode `resume(self)` de la classe `TrainingThread` remet `paused` à `False` pour relancer la boucle.

```
def resume(self):
```

La classe `TrainingGUI` est l'interface graphique d'entrainement. Elle gère les trois onglets, les contrôles de configuration et la communication avec le thread d'entrainement.

```
class TrainingGUI:
```

La methode `__init__(self, root)` de la classe `TrainingGUI` est le constructeur. Elle initialise toutes les variables de l'application, charge l'état actuel depuis le disque avec `_load_current_state()` et construit l'interface avec `_build_interface()`.

```
def __init__(self, root):
```

La methode `_load_current_state(self)` de la classe `TrainingGUI` charge l'index de la génération la plus récente et les poids correspondants depuis le disque. En cas d'erreur de lecture elle replie sur les poids par défaut.

```
def _load_current_state(self):
```

La methode `_build_interface(self)` de la classe `TrainingGUI` crée le Notebook Tkinter avec ses trois onglets et délègue la construction de chacun aux méthodes dédiées.

```
def _build_interface(self):
```

La methode `_build_training_tab(self, parent)` de la classe `TrainingGUI` construit l'onglet Entraînement avec le label de génération, les spinboxes de configuration et les boutons Démarrer, Pause et Arrêter à gauche, et la barre de progression avec le journal scrollable à droite.

```
def _build_training_tab(self, parent):
```

La methode `_build_match_tab(self, parent)` de la classe `TrainingGUI` construit l'onglet Match avec le spinbox du nombre de parties, les deux blocs de chargement des poids A et B et le bouton Démarrer le Match.

```
def _build_match_tab(self, parent):
```

La methode `_build_stats_tab(self, parent)` de la classe `TrainingGUI` construit l'onglet Statistiques avec une zone de texte scrollable et appelle `update_stats_display()` pour la remplir dès l'ouverture.

```
def _build_stats_tab(self, parent):
```

La methode `start_training(self)` de la classe `TrainingGUI` lit les valeurs des spinboxes, active les boutons Pause et Stop, désactive Démarrer, crée et lance un `TrainingThread` en mode training.

```
def start_training(self):
```

La methode `start_match(self)` de la classe `TrainingGUI` lit le nombre de parties, vide le journal de match, crée et lance un `TrainingThread` en mode match.

```
def start_match(self):
```

La methode `pause_training(self)` de la classe `TrainingGUI` bascule entre pause et reprise. Elle met à jour le texte du bouton et appelle `thread.pause()` ou `thread.resume()` selon l'état actuel.

```
def pause_training(self):
```

La methode stop_training(self) de la classe TrainingGUI appelle thread.stop() pour demander l'arrêt de l'entrainement.

```
def stop_training(self):
```

La methode training_finished(self) de la classe TrainingGUI est appelée par le thread en fin d'execution. Elle remet les flags à zéro et programme _finish_ui() dans le thread principal via root.after(0).

```
def training_finished(self):
```

La methode _finish_ui(self) de la classe TrainingGUI remet les boutons dans leur état initial, met à jour le label de génération et rafraichit l'onglet statistiques.

```
def _finish_ui(self):
```

La methode update_log(self, msg) de la classe TrainingGUI ajoute une ligne dans le journal approprié selon le mode actuel. Elle utilise root.after(0) pour s'assurer que la mise à jour se fait dans le thread principal Tkinter car seul lui peut modifier les widgets. Elle contient la fonction interne _u() qui effectue l'insertion.

```
def update_log(self, msg):
```

La methode update_progress(self, cur, tot) de la classe TrainingGUI met à jour la barre de progression et son label. Elle utilise root.after(0) avec la fonction interne _u() pour la même raison que update_log.

```
def update_progress(self, cur, tot):
```

La methode add_training_stats(self, stats) de la classe TrainingGUI ajoute un dictionnaire de statistiques à training_history. Elle est appelée par le thread après chaque itération.

```
def add_training_stats(self, stats):
```

La methode update_stats_display(self) de la classe TrainingGUI reecrit entièrement le contenu de l'onglet Statistiques avec le résumé global et les 20 dernières itérations.

```
def update_stats_display(self):
```

La methode load_weights(self, side) de la classe TrainingGUI ouvre un filedialog pour choisir un fichier JSON de poids et les assigne au joueur A ou B selon le parametre side.

```
def load_weights(self, side):
```

La methode use_best_weights(self, side) de la classe TrainingGUI charge automatiquement les meilleurs poids actuels et les assigne au joueur A ou B.

```
def use_best_weights(self, side):  
    [...]  
  
    def load() :  
  
        [...]
```

La methode use_generation(self, side) de la classe TrainingGUI ouvre une petite fenêtre avec un spinbox pour choisir un numéro de génération. Elle contient la fonction interne load() qui charge les poids de la génération choisie et les assigne au joueur A ou B.

```
def use_generation(self, side):
```

La fonction main() crée la fenêtre Tkinter, instancie TrainingGUI et lance la boucle principale.

```
def main():
```

Le groupe interfaces graphiques est composé de gui.py et training_gui.py qui sont deux interfaces complètement séparées, chacune avec son propre point d'entrée.

gui.py s'appuie entièrement sur ai.py pour faire jouer l'IA. Quand c'est au tour de l'IA, gui.py appelle _analyze_position qui lance negamax sur chaque coup légal, récupère les scores et les affiche dans le panneau AnalysisPanel. Le joueur humain interagit via les boutons de l'échiquier, gui.py traduit ces clics en coups via engine_chess.py et met à jour l'affichage.

training_gui.py fonctionne différemment. Elle ne s'appuie pas sur gui.py mais appelle directement ai.py via choose_best_move_timed pour faire jouer les parties d'entraînement. Toute la logique lourde tourne dans un TrainingThread séparé pour ne pas bloquer l'interface. Ce thread communique avec l'interface uniquement via root.after() pour mettre à jour les labels et le journal, car Tkinter n'accepte pas qu'un thread secondaire touche aux widgets directement

C) fichier qui gère les règles :

--fichier engine_chess.py :

Utilité et fonctionnement :

`engine_chess.py` est le fichier qui permet de relier la bibliothèque `python-chess` à l'interface graphique du programme. Son rôle est de permettre au GUI d'utiliser `python-chess` tout en gardant la même structure que l'ancien moteur d'échecs. Le reste du programme peut donc fonctionner normalement sans modification car ce fichier convertit les données dans le format attendu.

Ce fichier sert de couche d'adaptation entre deux systèmes. Le moteur interne utilise des objets comme `chess.Board` et `chess.Move` alors que l'interface graphique utilise une grille 8x8 d'entiers, des coordonnées ligne-colonne et des chaînes de caractères pour les coups. `engine_chess.py` transforme les données dans les deux sens afin que le moteur et l'interface puissent communiquer correctement.

Ce fichier ne contient pas d'intelligence artificielle. Il sert uniquement à manipuler les données du jeu d'échecs et à les convertir. Le fonctionnement repose principalement sur deux classes. `Move` représente un coup compatible avec l'interface graphique tout en gardant une correspondance avec `chess.Move`. `GameState` représente un état du jeu compatible avec l'interface graphique et contient un objet `chess.Board` interne.

Les constantes `PAWN`, `KNIGHT`, `BISHOP`, `ROOK`, `QUEEN` et `KING` représentent les différents types de pièces avec les mêmes valeurs que `python-chess`. Le dictionnaire `_SYMBOLS` associe chaque type de pièce et chaque couleur à un symbole Unicode afin de permettre l'affichage des pièces dans l'interface

Explications du code :

La fonction `piece_symbol(piece)` retourne le symbole Unicode correspondant à une pièce afin de permettre son affichage dans l'interface graphique. Elle accepte une pièce `python-chess`, un entier signé correspondant à l'ancien moteur ou une case vide et renvoie le symbole adapté.

```
def piece_symbol(piece) -> str:
```

La classe `Move` représente un coup compatible avec l'interface graphique. Elle contient les coordonnées de départ et d'arrivée du coup ainsi que les informations nécessaires pour gérer les promotions, les roques et les prises en passant. Elle contient aussi un objet `chess.Move` interne afin de pouvoir transmettre directement le coup au moteur `python-chess`.

```
class Move:
```

La methode from_chess(cm, board) construit un objet Move à partir d'un objet chess.Move. Elle convertit les coordonnées du moteur interne vers les coordonnées utilisées par l'interface graphique et détecte automatiquement les promotions, les roques et les prises en passant.

```
def from_chess(cm: chess.Move, board: chess.Board) -> "Move":
```

La methode to_chess(self) convertit un objet Move en objet chess.Move afin que le moteur interne puisse comprendre le coup choisi par l'utilisateur dans l'interface graphique. Si l'objet chess.Move existe déjà, il est réutilisé directement.

```
def to_chess(self) -> chess.Move:
```

La classe GameState représente un état du jeu compatible avec l'interface graphique. Elle contient un objet chess.Board interne et une liste contenant l'historique des coups joués. Elle permet au GUI d'accéder au plateau sous forme de grille simple tout en utilisant python-chess en arrière-plan.

```
class GameState:
```

La methode init(self, board, turn) initialise l'état du jeu. Elle crée un plateau initial si aucun plateau n'est fourni et prépare la liste qui stocke les coups joués pendant la partie.

```
def __init__(self, board: Optional[chess.Board] = None, turn: int = 1)
```

La propriété board construit dynamiquement une grille 8x8 d'entiers à partir du plateau interne. Les entiers positifs représentent les pièces blanches, les entiers négatifs représentent les pièces noires et la valeur 0 représente une case vide. Cette grille est utilisée directement par l'interface graphique pour afficher le plateau.

```
@property  
def board(self) -> List[List[int]]:
```

La propriété turn retourne le joueur actif sous forme de 1 pour les blancs ou -1 pour les noirs. Elle permet aussi de modifier le joueur actif si nécessaire en mettant à jour le plateau interne.

```
@property  
def turn(self) -> int:
```

La propriété `move_history` retourne la liste des coups joués en notation UCI. Elle permet notamment de sauvegarder ou rejouer une partie.

```
@property  
def move_history(self) -> List[str]:
```

La méthode `get_board(self)` retourne l'objet `chess.Board` interne afin que les autres parties du programme puissent accéder directement au moteur python-chess lorsque c'est nécessaire.

```
def get_board(self) -> chess.Board:
```

La méthode `copy(self)` crée une copie complète de l'état du jeu en copiant le plateau et l'historique des coups. Elle permet de tester des coups ou de simuler des positions sans modifier la partie en cours.

```
def copy(self) -> "GameState":
```

La méthode `king_pos(color)` retourne la position du roi de la couleur demandée sous forme de coordonnées ligne et colonne. Elle est notamment utilisée pour certaines vérifications comme les échecs.

```
def king_pos(self, color: int) -> Tuple[int, int]:
```

La fonction `initial_board()` retourne un plateau initial sous forme d'objet `chess.Board` afin de commencer une nouvelle partie.

```
def initial_board() -> chess.Board:
```

La fonction `legal_moves(state)` retourne la liste des coups légaux sous forme d'objets `Move` en utilisant les coups générés par python-chess. Cette liste est utilisée par le GUI pour savoir quels coups sont possibles.

```
def legal_moves(state: GameState) -> List[Move]:
```

La fonction `apply_move_inplace(state, move)` applique un coup directement sur l'état du jeu après avoir vérifié que le coup est légal. Le coup est ensuite ajouté à l'historique de la partie.

```
def apply_move_inplace(state: GameState, move) -> None:
```

La fonction `make_move(state, move)` retourne un nouvel état du jeu après application du coup sans modifier l'état original. Elle est utilisée lorsque le programme doit tester des coups.


```
def make_move(state: GameState, move) -> GameState:
```

La fonction `is_checkmate(state)` vérifie si la position actuelle correspond à un échec et mat et permet donc de détecter la fin d'une partie.

```
def is_checkmate(state: GameState) -> bool:
```

La fonction `is_stalemate(state)` vérifie si la position correspond à un pat et permet de détecter une partie nulle.

```
def is_stalemate(state: GameState) -> bool:
```

La fonction `in_check(state, color)` vérifie si une couleur donnée est en échec en utilisant les informations du plateau interne.

```
def in_check(state: GameState, color: int) -> bool:
```

La fonction `has_legal_moves(state)` vérifie s'il existe au moins un coup légal dans la position actuelle et permet de savoir si le joueur peut encore jouer.

```
def has_legal_moves(state: GameState) -> bool:
```

La fonction `move_to_uci(move)` convertit un coup en chaîne de caractères au format UCI afin de pouvoir l'enregistrer ou l'afficher.

```
def move_to_uci(move) -> str:
```

La fonction `uci_to_move(state, uci)` convertit une chaîne UCI en objet `Move` après avoir vérifié que le coup correspond à un coup légal dans la position actuelle.

```
def uci_to_move(state: GameState, uci: str) -> Move:
```

La fonction `state_to_fen(state)` convertit un état du jeu en notation FEN afin de pouvoir sauvegarder une position complète.

```
def state_to_fen(state: GameState) -> str:
```

La fonction `fen_to_state(fen)` crée un état du jeu à partir d'une notation FEN afin de pouvoir charger une position sauvegardée

```
def fen_to_state(fen: str) -> GameState:
```

Ce groupe de fichiers contient le module chargé de gérer les règles du jeu d'échecs. Il est composé uniquement du fichier `engine_chess.py`, qui s'occupe de représenter le plateau, les coups possibles et l'évolution d'une partie en respectant les règles officielles des échecs

Ce module utilise la bibliothèque `python-chess` pour gérer automatiquement les règles du jeu comme les déplacements des pièces, les coups légaux, les échecs, les mats ou les situations de pat. Le fichier `engine_chess.py` sert d'intermédiaire entre cette bibliothèque et l'interface graphique du programme. Il convertit les informations du moteur interne vers un format simple que les autres fichiers peuvent utiliser, notamment une grille 8×8 et des coordonnées ligne-colonne.

Ce groupe de fichiers permet donc de gérer toute la logique du jeu d'échecs sans contenir d'intelligence artificielle. Il fournit les fonctions nécessaires pour créer une partie, générer les coups légaux, appliquer un coup, vérifier la fin d'une partie et sauvegarder ou charger une position. Grâce à cette organisation, les autres parties du programme peuvent utiliser les règles du jeu sans avoir besoin de manipuler directement la bibliothèque `python-chess`.

D) fichier de système d'entraînement :

--fichier `weights.py` :

Utilité et fonctionnement :

`weights.py` est le fichier qui définit et gère tous les paramètres numériques de la fonction d'évaluation. C'est lui qui rend l'IA entraînable : plutôt que d'avoir des valeurs fixes décidées à l'avance, tous les coefficients utilisés par `evalu.py` sont regroupés ici et peuvent être modifiés par le système d'entraînement. Il gère aussi toute la partie sauvegarde et chargement de ces paramètres sur le disque.

Le fonctionnement est organisé en deux parties. La première est la définition des données : `DEFAULT_WEIGHTS` contient les 40 valeurs de départ, `WEIGHT_BOUNDS` définit les limites autorisées pour chacune et `PARAM_GROUPS` les regroupe par thème pour que les mutations soient ciblées. La deuxième partie est un ensemble de fonctions utilitaires pour lire et écrire ces paramètres dans des fichiers JSON, gérer les générations et savoir quelle est la meilleure version actuelle.

Explication du code :

-variables principales :

DEFAULT_WEIGHTS est la variable la plus importante du fichier. C'est un dictionnaire de 40 paramètres avec leurs valeurs de départ, organisés en plusieurs catégories. Les valeurs des pièces comme pv_pawn à 100 ou pv_queen à 900 définissent combien vaut chaque type de pièce en centipions. Les multiplicateurs de phase comme op_center à 1.2 ou eg_pawns à 1.4 définissent l'importance de chaque critère selon qu'on est en ouverture, milieu de jeu ou finale. C'est le point de départ de tout entraînement : une IA qui n'a jamais été entraînée utilisera exactement ces valeurs.

```
DEFAULT_WEIGHTS: Dict[str, float] = {  
    # Valeurs des pièces (centipawns)  
    "pv_pawn": 100.0,  
    "pv_knight": 320.0,  
    "pv_bishop": 330.0,  
    "pv_rook": 500.0,  
    "pv_queen": 900.0,  
    [...]
```

WEIGHT_BOUNDS est le dictionnaire qui associe à chaque paramètre un tuple contenant sa valeur minimale et maximale autorisée. Par exemple pv_pawn est limité entre 50 et 200, et pv_queen entre 600 et 1200. Sans ces bornes une mutation pourrait donner des valeurs absurdes comme un pion qui vaut plus qu'une dame, ce qui rendrait l'IA complètement inutilisable.

```
WEIGHT_BOUNDS: Dict[str, tuple] = {  
    "pv_pawn": (50.0, 200.0),  
    "pv_knight": (200.0, 500.0),  
    "pv_bishop": (200.0, 500.0),  
    "pv_rook": (300.0, 700.0),  
    "pv_queen": (600.0, 1200.0),  
  
    "op_material": (0.5, 2.0),  
    "op_center": (0.0, 3.0),  
    "op_development": (0.0, 3.0),  
    "op_king_safety": (0.0, 3.0),  
    "op_mobility": (0.0, 2.0),  
    "op_pawns": (0.0, 2.0),  
    [...]
```

PARAM_GROUPS est le dictionnaire qui organise les 40 paramètres en 9 groupes thématiques : piece_values, opening, middlegame, endgame, king_safety, pawns, mobility, center et phase. Quand l'algorithme d'entraînement veut muter les poids, il choisit un groupe entier plutôt que des paramètres au hasard. Ça garantit que les mutations sont cohérentes : par exemple si on mute le groupe opening tous les multiplicateurs d'ouverture évoluent ensemble ce qui a plus de sens stratégiquement.

```
PARAM_GROUPS = {
    "piece_values": ["pv_pawn", "pv_knight", "pv_bishop", "pv_rook", "pv_queen"],
    "opening": [k for k in DEFAULT_WEIGHTS if k.startswith("op_")],
    "middlegame": [k for k in DEFAULT_WEIGHTS if k.startswith("mg_")],
    "endgame": [k for k in DEFAULT_WEIGHTS if k.startswith("eg_")],
    "king_safety": [k for k in DEFAULT_WEIGHTS if k.startswith("ks_")],
    "pawns": [k for k in DEFAULT_WEIGHTS if k.startswith("pawn_")],
    "mobility": [k for k in DEFAULT_WEIGHTS if k.startswith("mobility_")],
    "center": [k for k in DEFAULT_WEIGHTS if k.startswith("center_")],
    "phase": [k for k in DEFAULT_WEIGHTS if k.startswith("phase_")],
}
```

-fonction :

La fonction project_root() retourne le chemin absolu du répertoire racine du projet en remontant d'un niveau depuis le répertoire de weights.py. Elle est utilisée par les autres fonctions pour construire les chemins des fichiers.

```
def project_root() -> str:
```

La fonction generations_dir() retourne le chemin du dossier generations où sont stockés tous les fichiers de poids de chaque génération.

```
def generations_dir() -> str:
```

La fonction best_path() retourne le chemin du fichier best.json qui contient uniquement un pointeur vers le meilleur fichier de poids actuellement connu.

```
def best_path() -> str:
```

La fonction gen_path(n) retourne le chemin du fichier de poids d'une génération précise, par exemple weights_gen_5.json pour la génération 5.

```
def gen_path(n: int) -> str:
```

La fonction `clamp_weights(w)` prend un dictionnaire de poids et ramène chaque valeur dans ses bornes autorisées selon `WEIGHT_BOUNDS`. Elle est appelée après chaque mutation pour garantir que les valeurs restent dans des plages raisonnables.

```
def clamp_weights(w: Dict[str, float]) -> Dict[str, float]:
```

La fonction `ensure_defaults()` garantit l'existence de tous les fichiers nécessaires au démarrage. Elle crée le dossier `generations` si besoin, crée `weights_gen_0.json` avec les valeurs par défaut s'il est absent et crée `best.json` pointant vers `gen_0` s'il est absent. Elle est appelée au démarrage de tous les modules qui utilisent le système de poids.

```
def ensure_defaults() -> None:
```

La fonction `save_gen(path, gen_id, weights, name)` sauvegarde un dictionnaire de poids dans un fichier JSON structuré contenant la version, le nom, la date de création et les poids arrondis à 4 décimales.

```
def save_gen(path: str, gen_id: int, weights: Dict[str, float],  
             name: Optional[str] = None) -> None:
```

La fonction `load_gen(path)` charge un fichier de poids JSON et retourne les poids fusionnés avec les valeurs par défaut. Si un paramètre est absent du fichier il prend sa valeur par défaut, ce qui garantit la compatibilité avec les anciennes sauvegardes si on ajoute de nouveaux paramètres.

```
def load_gen(path: str) -> Dict[str, float]:
```

La fonction `get_best_weights()` charge et retourne les meilleurs poids actuels en lisant `best.json` pour trouver le chemin du fichier puis en le chargeant avec `load_gen`. Si le fichier pointé n'existe plus elle replie sur `gen_0`.

```
def get_best_weights() -> Dict[str, float]:
```

La fonction `get_best_file()` retourne simplement le chemin du meilleur fichier de poids actuel sans charger son contenu.

```
def get_best_file() -> str:
```

La fonction `set_best(weights_file)` écrit dans `best.json` le chemin absolu du nouveau meilleur fichier de poids. Elle est appelée après chaque candidat accepté pour mettre à jour le pointeur.

```
def set_best(weights_file: str) -> None:
```

La fonction `latest_generation_index()` parcourt les fichiers `weights_gen_0.json`, `weights_gen_1.json` et ainsi de suite en incrementant jusqu'à ne plus en trouver. Elle retourne le numéro de la génération la plus récente

```
def latest_generation_index() -> int:
```

--fichier `self_train.py` :

Utilité et fonctionnement :

`elfplay_train.py` est la version ligne de commande du système d'entraînement. Il fait exactement la même chose que `training_gui.py` mais sans interface graphique, ce qui le rend idéal pour des entraînements longs en arrière plan. Son principe est simple : charger les meilleurs poids actuels, en créer une version légèrement modifiée, les faire s'affronter sur plusieurs parties, et garder la version modifiée si elle s'avère meilleure. Chaque résultat est enregistré dans un fichier d'historique pour pouvoir suivre l'évolution de l'IA dans le temps.

Le fonctionnement se déroule en boucle. A chaque itération on mute les poids du champion pour créer un candidat, on lance un match entre les deux en alternant les couleurs pour une évaluation équitable, et si le candidat dépasse le seuil de `winrate` il devient le nouveau champion et est sauvegardé comme nouvelle génération. Tout est paramétrable via un fichier `config.json` : nombre de parties par évaluation, temps par coup, nombre d'itérations et intensité des mutations.

Explication du code :

Variables principales : La variable `PROJECT_ROOT` est le chemin absolu du répertoire racine du projet. Elle est calculée automatiquement depuis l'emplacement du fichier et est utilisée par toutes les autres fonctions pour construire les chemins des fichiers de configuration et d'historique.

```
PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__)))
```

La variable `CONFIG_PATH` est le chemin vers le fichier `config.json` qui contient tous les paramètres de la session d'entraînement. Si ce fichier n'existe pas le programme utilise des valeurs par défaut raisonnables, ce qui permet de lancer un entraînement sans configuration préalable

```
CONFIG_PATH = os.path.join(PROJECT_ROOT, "config.json")
```

Fonction :

La fonction `load_config()` charge et retourne le fichier `config.json` sous forme de dictionnaire. Si le fichier n'existe pas elle retourne un dictionnaire vide et le programme utilisera les valeurs par défaut définies dans `main()`.

```
def load_config() -> dict:
```

La fonction `now()` retourne l'horodatage actuel sous forme de chaîne ISO 8601. Elle est utilisée pour horodater chaque entrée dans le fichier d'historique et savoir exactement quand chaque itération a eu lieu.

```
def now() -> str :
```

La fonction `mutate_weights(w, mutation_strength)` crée une version mutée des poids. Elle choisit aléatoirement un groupe de paramètres dans `PARAM_GROUPS`, mute 1 à 3 paramètres de ce groupe avec une amplitude proportionnelle à leur échelle et à `mutation_strength`, et retourne le nouveau dictionnaire de poids ainsi que le nom du groupe muté et la liste des clés modifiées. Ces informations supplémentaires permettent de les afficher dans le terminal et de les enregistrer dans l'historique.

```
def mutate_weights(w: dict, mutation_strength: float = 1.0) -> tuple[dict, str, list]:
```

La fonction `play_game(weights_white, weights_black, movetime_ms, max_plies)` joue une partie complète entre deux jeux de poids. Elle injecte les poids du joueur actif avant chaque coup via `set_active_weights`, s'arrête en cas de fin de partie ou après `max_plies` demi coups, et retourne +1 si les blancs gagnent, -1 si les noirs gagnent et 0 pour toute nulle.

```
def play_game(weights_white: dict, weights_black: dict,  
               movetime_ms: int, max_plies: int) -> int:
```

La fonction `evaluate_match(best, cand, n_games, movetime_ms, max_plies)` fait jouer `n_games` parties entre le champion et le candidat en alternant les couleurs. Elle affiche le score en temps réel dans le terminal après chaque partie et retourne le winrate du candidat ainsi que le dictionnaire de statistiques avec le nombre de victoires, nulles et défaites.

```
def evaluate_match(best: dict, cand: dict,  
                  n_games: int, movetime_ms: int,  
                  max_plies: int) -> tuple[float, dict]:
```

La fonction `main()` est la boucle principale du programme. Elle commence par charger la configuration et les meilleurs poids actuels, puis pour chaque itération elle mute les poids, affiche les modifications effectuées, lance le match avec `evaluate_match`, accepte ou rejette le candidat selon le `winrate`, sauvegarde si accepté, et enregistre le résultat complet dans le fichier `history_selfplay.jsonl` pour garder une trace de toute l'évolution de l'IA

```
def main():
```

--fichier `reset_training.py` :

Utilité et fonctionnement : `reset_training.py` est un petit script utilitaire qui permet de remettre à zéro toutes les données d'entraînement. Concrètement il supprime le dossier `generations` qui contient tous les fichiers de poids générés pendant l'entraînement, puis recrée les fichiers de départ avec les valeurs par défaut. Il est utilisé quand on veut repartir d'une IA vierge sans garder les générations précédentes.

Le fonctionnement est simple. Avant de supprimer quoi que ce soit il propose de faire une sauvegarde automatique en copiant le dossier `generations` dans un nouveau dossier horodaté, ce qui permet de ne pas perdre définitivement le travail effectué. On peut désactiver cette sauvegarde en passant l'argument `--no-backup` en ligne de commande.

Explication du code :

Variables principales :

La variable `gen_dir` contient le chemin du dossier d'entraînement, c'est à dire `generations`. C'est le dossier qui sera supprimé lors de la réinitialisation.

```
gen_dir = "generations"
```

La variable `timestamp` contient l'horodatage au format `YYYYMMDD_HHMMSS` généré au moment de l'appel. Il est utilisé pour nommer le dossier de sauvegarde de manière unique, ce qui permet de faire plusieurs réinitialisations successives sans écraser les sauvegardes précédentes.

```
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
```


La variable `backup_dir` contient le nom du dossier de sauvegarde construit à partir du timestamp, par exemple `generations_backup_20241201_153000`.

```
backup_dir = f"generations_backup_{timestamp}"
```

Fonction :

La fonction `reset_training(backup)` est la fonction principale du fichier. Elle vérifie d'abord si le dossier `generations` existe, et si ce n'est pas le cas elle s'arrête en affichant un message. Si `backup` vaut `True` elle copie tout le dossier dans un nouveau dossier horodaté avant de le supprimer. Elle supprime ensuite le dossier avec `shutil.rmtree` puis appelle `ensure_defaults()` de `weights.py` pour recréer les fichiers de départ proprement.

```
def reset_training(backup=True):
```

Le bloc `if name == "main"` est le point d'entrée du script en ligne de commande. Il lit les arguments passés par l'utilisateur via `sys.argv` et appelle `reset_training` avec `backup=False` si l'argument `--no-backup` est présent, ou `backup=True` par défaut si aucun argument n'est fourni.

```
if __name__ == "__main__":
```

--fichier `plot_progress.py`

Utilité et fonctionnement :

`plot_progress.py` est un petit outil de visualisation. Il lit le fichier d'historique de l'entraînement `history.jsonl` et génère un graphique qui montre l'évolution du winrate au fil des générations. C'est utile pour voir d'un coup d'oeil si l'IA progresse bien ou si elle stagne. Le graphique est sauvegardé automatiquement dans le dossier `generations` sous le nom `progress.png`.

Le fonctionnement est simple : il parcourt le fichier d'historique ligne par ligne, extrait le numéro de génération et le winrate de chaque entrée, et trace une courbe avec `matplotlib`. Si le fichier d'historique n'existe pas ou est vide il affiche un message d'erreur et s'arrête

Explication du code :

La variable `HIST` contient le chemin vers le fichier `history.jsonl`. Il est construit automatiquement depuis l'emplacement du fichier pour fonctionner peu importe où le projet est installé.

```
HIST = os.path.join(os.path.dirname(__file__), "..", "generations", "history.jsonl")
```

La fonction `main()` est le seul point d'entrée du fichier. Elle vérifie d'abord que le fichier d'historique existe, puis le parcourt ligne par ligne en extrayant le numéro de génération depuis le nom du fichier de poids et le winrate associé. Elle trace ensuite la courbe avec `matplotlib` et sauvegarde le graphique en png avec une résolution de 150 dpi

```
def main():
```

Le système d'entraînement est composé de quatre fichiers qui forment un ensemble cohérent : `weights.py`, `selfplay_train.py`, `training_gui.py` et `reset_training.py`. Chacun a un rôle précis et ils s'appuient les uns sur les autres.

`weights.py` est la base de tout le système. Sans lui rien ne fonctionnerait car c'est lui qui définit les 40 paramètres que les autres fichiers vont faire évoluer. Il est utilisé par tous les autres fichiers du système d'entraînement et aussi par `evalu.py` qui a besoin de ces valeurs pour évaluer les positions.

`selfplay_train.py` et `training_gui.py` font exactement la même chose mais de deux façons différentes. Le premier est destiné aux entraînements longs en arrière plan sans avoir besoin de regarder l'écran, le second offre une interface graphique pour suivre la progression en temps réel et lancer des matchs comparatifs. Dans les deux cas la logique est identique : muter les poids, faire jouer les deux versions l'une contre l'autre, garder la meilleure. Ils s'appuient tous les deux sur `weights.py` pour charger et sauvegarder les générations, et sur `ai.py` pour faire jouer les parties.

`reset_training.py` est le plus simple des quatre. Il n'intervient pas dans le processus d'entraînement lui même mais permet de tout remettre à zéro proprement quand on veut recommencer depuis le début. Il s'appuie lui aussi sur `weights.py` pour recréer les fichiers de départ. Ce qui rend ce système intéressant c'est qu'il est entièrement autonome une fois lancé. L'IA joue contre elle même, évalue ses propres résultats et améliore ses paramètres sans aucune intervention humaine. Au bout de suffisamment de générations l'IA a trouvé par elle même des valeurs plus efficaces que celles définies à la main au départ, un peu comme un joueur qui s'améliore en s'entraînant régulièrement.

E) fichier Main : `main.py`, `main_train.py`

--fichier `main_train.py`

Utilité et fonctionnement :

main.py est le point d'entrée du jeu. C'est le fichier qu'on lance pour ouvrir l'interface de jeu et commencer une partie. Il ne contient quasiment pas de logique propre : son seul rôle est de configurer le chemin du projet, importer ChessApp depuis gui.py et lancer la fenêtre Tkinter. Tout le reste est délégué à gui.py

Explication du code :

Le bloc if name == "main" est le seul point d'entrée du fichier. Il affiche quelques informations dans le terminal pour rappeler les fonctionnalités disponibles, crée la fenêtre Tkinter avec tk.Tk(), instancie ChessApp en lui passant cette fenêtre, et lance la boucle principale avec root.mainloop() qui maintient l'interface ouverte jusqu'à ce que l'utilisateur la ferme

```
if __name__ == "__main__":
    print("🎮 Lancement de l'interface avec analyse avancée...")
    print()
    print("💎 Fonctionnalités :")
    print(" - 📊 Panneau d'analyse en temps réel")
    print(" - 🏆 Top 5 meilleurs coups affichés")
    print(" - ✓ Coup choisi mis en évidence")
    print(" - 📐 Évaluation de position")
    print(" - ⚡ Statistiques de performance")
    print()
    print("💡 Astuce : Augmentez le délai (800-1500ms) pour bien observer l'analyse")
    print()

    root = tk.Tk()
    app = ChessApp(root)
    root.mainloop()
```

--fichier main_training.py :

Utilité et fonctionnement :

main_train.py est le point d'entrée de l'interface d'entraînement. C'est le fichier qu'on lance pour ouvrir l'interface qui permet de gérer l'entraînement de l'IA. Comme main.py il ne contient quasiment pas de logique propre : son seul rôle est de configurer le chemin du projet, importer TrainingGUI depuis training_gui.py et lancer la fenêtre Tkinter. Tout le reste est délégué à training_gui.py

Explication du code :

Le bloc `if name == "main"` est le seul point d'entrée du fichier. Il affiche dans le terminal un rappel des trois modes disponibles : entraînement automatique par mutation, match entre deux versions de poids et consultation des statistiques. Il crée ensuite la fenêtre Tkinter, instancie `TrainingGUI` et lance la boucle principale avec `root.mainloop()`.

```
if __name__ == "__main__":
    print("🌀 Lancement de l'interface d'entraînement...")
    print()
    print("🌟 Fonctionnalités :)")
    print("🎓 Mode Entraînement:")
    print("    - Évolution automatique par mutation")
    print("    - Self-play entre candidat et champion")
    print("    - Suivi de la progression en temps réel")
    print()
    print("✂️ Mode Match:")
    print("    - Comparer deux versions de poids")
    print("    - Charger depuis fichiers ou générations")
    print("    - Statistiques détaillées")
    print()
    print("📊 Statistiques:")
    print("    - Historique complet")
    print("    - Taux d'acceptation")
    print("    - Évolution des performances")
    print()

    root = tk.Tk()
    app = TrainingGUI(root)
    root.mainloop()
```

Le groupe main est composé de deux fichiers : `main.py` et `main_train.py`. Ils sont les plus courts et les plus simples du projet mais ils sont indispensables car ce sont eux qu'on lance pour utiliser le projet.

Leur rôle est uniquement d'être des portes d'entrée. `main.py` ouvre le jeu, `main_train.py` ouvre l'interface d'entraînement. Aucun des deux ne contient de logique métier : ils se contentent de configurer le chemin du projet, d'importer la bonne classe et de lancer la boucle Tkinter. Tout le travail réel est fait par les fichiers qu'ils importent.

Ce qui les distingue l'un de l'autre c'est simplement leur destination. `main.py` est destiné à l'utilisateur qui veut jouer, `main_train.py` est destiné à celui qui veut améliorer l'IA. En pratique on ne lance jamais les deux en même temps car ils partagent les mêmes fichiers de poids sur le disque et cela pourrait provoquer des conflits lors des sauvegardes.

3° CONCLUSION, AXE D'AMELIORATION, DIFFICULTEES

Axe d'amélioration :

Le premier axe d'amélioration concerne la profondeur de recherche. Actuellement l'IA est limitée à quelques coups d'avance car augmenter la profondeur ralentit exponentiellement le calcul. Une piste serait d'implémenter un meilleur système de gestion du temps ou d'optimiser davantage l'élagage pour permettre d'explorer plus loin sans perdre en vitesse.

Le deuxième axe concerne le système d'entraînement. Actuellement les mutations sont aléatoires et on ne garde que les candidats qui gagnent suffisamment de parties. Ce processus est lent et peut rater de bonnes pistes. On pourrait s'inspirer davantage des algorithmes génétiques en gardant plusieurs champions en parallèle et en les faisant se croiser entre eux plutôt que de toujours muter un seul champion.

Le troisième axe serait d'enrichir la fonction d'évaluation. Elle prend déjà en compte beaucoup de critères mais il en manque certains importants comme la coordination des pièces, les cases faibles dans la structure de pions ou les tours sur les colonnes ouvertes. Plus l'évaluation est précise moins l'IA a besoin de chercher loin pour jouer correctement.

Enfin on pourrait améliorer le livre d'ouverture qui est assez limité. Actuellement il ne contient qu'une vingtaine de lignes. L'ajouter de centaines de variantes connues permettrait à l'IA de mieux gérer les débuts de partie et d'éviter les pièges d'ouverture classiques

Difficultés rencontrées :

La plus grande difficulté du projet a été la mise en place de l'algorithme negamax avec toutes ses optimisations. Comprendre et implémenter correctement l'élagage alpha beta a demandé beaucoup de travail car le moindre bug dans les signes des scores ou dans la gestion des bornes alpha et beta provoque des coups complètement aberrants sans qu'on comprenne immédiatement pourquoi. La convention negamax ou le score est toujours du point de vue du joueur actif et doit être inversé à chaque niveau de récursion est particulièrement difficile à maîtriser.

La table de transposition a aussi été un point délicat. Il faut gérer correctement les trois types d'entrées (exact, lower bound, upper bound) et s'assurer que la profondeur stockée est suffisante pour pouvoir réutiliser le résultat. Une erreur ici peut rendre l'IA plus faible sans qu'on s'en rende compte facilement.

La gestion des threads dans `training_gui.py` a été une autre difficulté majeure. En python Tkinter n'est pas thread safe ce qui signifie qu'on ne peut pas modifier les widgets depuis un thread secondaire. Il a fallu comprendre et utiliser systématiquement `root.after()` pour toutes les mises à jour de l'interface depuis le thread d'entraînement, ce qui n'est pas intuitif au premier abord.

Enfin la conversion de coordonnées entre le système de python chess (cases numérotées de 0 à 63 depuis le bas) et le système du GUI (grille ligne colonne depuis le haut) a été une source de bugs récurrente. La formule $7 - \text{rang}$ pour inverser les lignes semble simple mais elle doit être appliquée de manière cohérente partout dans le code sous peine d'afficher les pièces au mauvais endroit ou de jouer des coups invalides.

Conclusion :

Ce projet nous a permis de construire un moteur d'échecs complet avec une intelligence artificielle capable de jouer et de s'améliorer par elle-même. En partant d'un simple affichage de plateau on est arrivé à un système complet avec un algorithme de recherche optimisé, une fonction d'évaluation paramétrable et un mécanisme d'entraînement automatique par évolution.

Ce qui rend ce projet particulièrement riche d'un point de vue informatique c'est qu'il touche à beaucoup de domaines différents. La récursivité et les algorithmes de recherche avec negamax, la programmation orientée objet avec les nombreuses classes du projet, la programmation parallèle avec les threads de l'entraînement, la gestion de fichiers avec le système de sauvegarde des générations et la conception d'interfaces graphiques avec Tkinter. Chacun de ces aspects nous a appris quelque chose de nouveau et nous a forcé à résoudre des problèmes concrets.

Le résultat est une IA qui joue à un niveau raisonnable et qui peut encore progresser si on la laisse s'entraîner suffisamment longtemps. C'est peut être ce qu'on retient le plus de ce projet : on n'a pas juste codé un programme qui joue aux échecs, on a codé un programme capable d'apprendre à mieux jouer tout seul, ce qui est une idée à la fois simple dans son principe et fascinante dans sa réalisation.